Karlsruhe University of Applied Sciences

Department of Computer Science and Business Information Systems

Business Information Systems

BACHELOR´S THESIS

Efficiency vs. Performance in Green AI: An Empirical Investigation of Energy Consumption, Carbon Footprint, and Predictive Accuracy of Classification Algorithms at Varying Dataset Sizes.

Author	Eron Qorri
Matriculation number	85383
Workplace	Hochschule Karlsruhe
First Advisor	Prof. Dr. rer. nat. Christine Preisach
Second Advisor	Prof. Dr. Reimar Hofmann
Closing Date	31 July, 2026

31 July, 2026

Chairman of the Examination Board

Contents

Chapter 1

Introduction

1.1 Problem statement and societal relevance

The current trend in machine learning is moving towards increasingly large models and datasets [1]. Although deep learning approaches often achieve the highest predictive accuracy, their energy demand and associated CO_2 emissions increase disproportionately [2, 3]. In an era marked by strict sustainability guidelines and rising energy costs, a critical question arises: At what point is a computationally "simple" model ecologically more viable than a highly complex one?

The primary objective of this thesis is to determine whether and when it is worthwhile to deploy more complex models, considering not only their predictive accuracy but also their associated energy consumption. To achieve this, a Logistic Regression, a Random Forest, an XGBoost model and a Multilayer Perceptron are compared under equal conditions across three datasets of varying sizes. This comparative analysis aims to provide a clearer understanding of the thresholds at which simpler models represent the better choice. Additionally, the study investigates how modifying a Multilayer Perceptron architecture, specifically varying the number of neurons and hidden layers, directly impacts energy consumption and CO_2 emissions.

To address the problem statement and achieve the outlined objectives, this thesis investigates the overarching question: *How does the relationship between energy consumption and predictive accuracy change for Logistic Regression, Random Forest, XGBoost, and Multilayer Perceptron when model complexity and dataset size are systematically varied?*

Specifically, the following sub-questions are examined:

- How do energy consumption (in kWh) and CO_2 emissions change when scaling a deep learning architecture (variation of neuron count)?
- Under which conditions (dataset size, model complexity) does the resource consumption of complex models exceed the achievable accuracy gain compared to simpler algorithms?

- How does the energy consumption differ between CPU-based training (Random Forest) and GPU-accelerated training (XGBoost, MLP) for the same classification task?

1.2 Motivation: When does a simpler model pay off?

1.3 Research questions and objectives

1.4 Structure of the Thesis

The remainder of this thesis is structured as follows: Chapter 2 provides the theoretical background, distinguishing between Green AI and Red AI, and introducing the relevant sustainability and classification metrics. Chapter 3 details the methodology, including dataset selection and the experimental hardware setup. Chapter 4 outlines the implementation process, focusing on data preprocessing, model training, and the logging of resource consumption. The empirical results are analyzed in Chapter 5, evaluating both performance and resource utilization across scaling dataset sizes. Chapter 6 discusses the findings, highlighting the economic and environmental implications. Finally, Chapter 7 concludes the thesis and provides actionable recommendations for sustainable machine learning practices.

$$S = \frac{F_1}{1 + \lambda_1 \cdot t_{\text{scaled}} + \lambda_2 \cdot c_{\text{scaled}}}$$

Chapter 2

Theoretical Background

2.1 Metrics

Chapter 3

Related Work

We also gathered several other characteristics of each data set, including number of categorical features (NCAT), number of continuous features (NCONT), proportion of features that are categorical (RCAT), number of target classes (NC), minimum single-class proportion (CMIN), and class imbalance (IC). The latter is a measure of the difference in actual and expected proportion of the least common target class (L) and its expected value (LE); i.e., I = (LE – L)/LE. These values are shown in Table 2.

Table 2: Additional Data Set Characteristics

Set	N_{CAT}	N_{CONT}	R_{CAT}	N_C	C_{MIN}	I_C
ANN	32	6	0.842	5	0.001	0.956
BCW	0	9	0.000	2	0.350	0.300
BMK	10	10	0.500	2	0.113	0.775
CAR	0	21	0.000	3	0.083	0.752
CVE	6	0	1.000	4	0.038	0.850
CVR	16	0	1.000	2	0.386	0.228
DCC	3	20	0.130	2	0.221	0.558
DRB	0	19	0.000	2	0.469	0.062
DRE	0	64	0.000	10	0.098	0.016
ILP	1	9	0.100	2	0.286	0.427
ION	0	34	0.000	2	0.359	0.282
LRE	0	16	0.000	26	0.037	0.046
MAG	0	10	0.000	2	0.322	0.357
MPE	3	77	0.038	8	0.092	0.259
MSH	22	0	1.000	2	0.482	0.036
OCD	0	5	0.000	2	0.231	0.538
PHW	30	0	1.000	2	0.443	0.114
QBD	0	41	0.000	2	0.337	0.325
SSG	0	3	0.000	2	0.208	0.585
WFR	0	24	0.000	4	0.060	0.760

During the MLP training experiments, several configurations were evaluated to optimize performance. Initially, the model was trained using only the CPU, which proved too slow for practical use. Switching to CUDA-based GPU training introduced a new issue: the default batch size of 128 provided by `scikit-learn` resulted in training times even slower than on CPU, due to the overhead of frequent small data transfers to the GPU.

Increasing the batch size to 8,192 significantly reduced training time. Further tuning of this parameter did not yield improvements, so this value was retained. Additionally, `pin_memory=True` was evaluated, which locks data in RAM to accelerate CPU-to-GPU transfers. However, this option also led to slower training, likely because the MLP architecture is not complex enough to benefit from the reduced transfer overhead. These experiments led to the final configuration used for all reported MLP results.

Chapter 4

Experimental Design

This chapter outlines the experimental design used to evaluate the ecological efficiency of different machine learning classification algorithms. It describes the datasets, models, evaluation metrics, hyperparameter tuning and measurement setup that form the basis of this empirical analysis.

4.1 Research Questions

The main research question that this study aims to answer is: *How does the trade-off between energy consumption and predictive performance vary across Random Forest, XGBoost and MLP models when the complexity of the models and the size of the dataset are systematically varied?* The following sub-questions will also be addressed in the course of this thesis:

- Under what conditions do the resource consumption of more complex models exceed the achievable accuracy gain over simpler baselines?
- How does energy consumption and CO2 output scale with increasing neural network complexity (number of layers and neurons)?
- How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?
- How does reducing the number of training instances on large datasets affect model accuracy and energy consumption across different algorithms?

To answer these questions multiple classification algorithms and datasets were used in the experimental phase of this empirical analysis.

4.2 Datasets

Dataset size is a primary driver of both computational demand and energy consumption during model training. To evaluate how this relationship varies across different scales, three

classification datasets were selected that deliberately span several orders of magnitude in size. Classification was chosen as the task type since it is well-established across all selected models and allows for direct comparison of predictive performance via standardized metrics such as accuracy and F1-score. The datasets are summarized in Table 4.1.

Table 4.1: Overview of datasets used in this analysis

Dataset	**Instances**	**Features**	**Classes**	**Scale**
Wine [4]	178	13	3	Small
Default of Credit Card Clients [5]	30,000	23	2	Medium
HIGGS [6]	11,000,000	28	2	Large

The *Wine* dataset contains 178 instances across 13 features, where each feature describes a chemical property of wines from the same region in Italy, derived from three different cultivars.

The *Default of Credit Card Clients* (hereafter: *Credit*) dataset contains 30,000 instances across 23 features and focuses on predicting payment default among credit card clients in Taiwan.

The *HIGGS* dataset is the largest dataset used in this study, comprising 11,000,000 instances across 28 features, seven of which are high-level features derived by physicists to help discriminate between the two classes [7]. The classification goal is to distinguish between signal events produced by Higgs bosons and background noise processes.

These three datasets vary significantly in size, enabling a systematic evaluation of how model performance and energy consumption scale across different levels of data complexity.

4.3 Models

Selecting models of varying complexity is central to answering the research questions of this study. A model that achieves competitive accuracy with significantly lower energy consumption represents a more ecologically efficient choice, but this trade-off can only be quantified if the comparison spans a meaningful range of complexity. For this reason, five classification models were selected, ranging from a linear baseline to tree-based ensemble methods and a neural network, covering low, medium, and high computational demand respectively.

Table 4.2: Overview of classification models used in this study

Model	**Type**	**Device**	**Role**
Logistic Regression [8]	Linear	CPU	Baseline
Random Forest [9]	Ensemble	CPU	Mid-complexity
XGBoost [10]	Gradient Boosting	CPU & GPU	Mid-complexity
Multilayer Perceptron	Neural Network	GPU	High-complexity

Remove the citations in the table as they are probably handled in 02

Logistic Regression serves as the lightweight baseline in this study. Despite its simplicity as a linear classifier, it often achieves competitive accuracy, making it well suited to quantifying whether the additional resource consumption of more complex models yields a meaningful gain in predictive performance.

Random Forest was selected as a mid-complexity representative of tree-based ensemble methods. As described in Section 2.x, it builds predictions by aggregating numerous decision trees trained on random subsets of the data. Relevant to this study, Random Forest does not support GPU-accelerated training and is therefore run exclusively on the CPU.

Building on the tree-based approach, XGBoost was chosen as a second mid-complexity model, with the key distinction that it also supports GPU-accelerated training. This makes it possible to directly compare the energy consumption and training time of CPU- and GPU-based tree ensemble methods under identical conditions, which is central to one of the research questions of this study.

The MLP was selected as the most complex model, representing the deep learning category. Its highly configurable architecture, particularly the number of hidden layers and neurons per layer, makes it well suited for the architectural complexity experiments conducted in this study, where these parameters are systematically varied to assess their impact on predictive performance and energy consumption. A more detailed description of the MLP architecture is provided in Section 2.x.

Together, the selected models cover a broad spectrum of algorithmic complexity, from a linear baseline to gradient boosting and deep learning, enabling a meaningful comparison of both predictive performance and energy efficiency across fundamentally different model types. The following section defines the metrics used to evaluate these dimensions.

4.4 Evaluation Metrics

To answer the research questions of this study, two categories of metrics were employed: classification performance and resource consumption. Accuracy and weighted F1-Score were selected to measure predictive performance, as they provide a standardized basis for comparing models across datasets of varying class distributions, as further described in Section 2.1. Energy consumption in kWh and CO_2 emissions in kg were chosen as the primary resource metrics, directly reflecting the ecological cost of model training. It should be noted that all experiments were conducted on a single hardware setup, which is described in detail in Section 5.2.

Training time in seconds was included as a secondary resource metric, as it captures computational demand independently of hardware-specific power draw. Furthermore, inference time was measured across all experimental setups to account for the operational footprint. In large-scale deployment scenarios, the cumulative energy demand of repeated

predictions can often rival or even exceed the initial training costs [11]. Together, these metrics enable a direct assessment of whether gains in predictive performance justify the associated increase in resource consumption.

Inspired by the efficiency framing proposed in Schwartz et al., a custom composite metric was developed for this study, named the *Efficiency-Weighted F1* (EWF1), defined as:

$$\text{EWF1} = \frac{F_1}{1 + \lambda_1 \cdot \hat{t} + \lambda_2 \cdot \hat{c}} \tag{4.1}$$

where $\hat{t}$ and $\hat{c}$ denote the min-max normalized training time and CO2 emissions per dataset respectively. The weights were set to $\lambda_1 = 0.5$ and $\lambda_2 = 1.0$, assigning greater importance to carbon emissions than training time, reflecting the ecological focus of this study. Higher scores indicate a more favorable trade-off between predictive performance and resource consumption.

It should be noted that both $\hat{t}$ and $\hat{c}$ are inherently hardware- and location-dependent, as training time varies with the underlying hardware configuration and CO2 emissions are tied to the carbon intensity of the local electricity grid [1]. Consequently, EWF1 scores reflect the ecological efficiency of the specific experimental setup used in this study and may not generalize directly to other hardware or energy infrastructures.

4.5 Validation Strategy

To obtain reliable and generalizable performance estimates, a cross-validation strategy was employed rather than a single train-test split [12]. A single split is highly sensitive to the random partitioning of the data, meaning that results may not be representative of true model performance, particularly on smaller datasets such as *Wine* [13]. Cross-validation mitigates this by evaluating each model across multiple data partitions, producing a more stable and unbiased estimate of generalization performance.

Specifically, K-Fold cross-validation was applied with $k = 5$ folds, a value widely recommended in the literature as a practical balance between computational cost and estimation stability [12]. Each model is trained and evaluated five times on non-overlapping subsets of the data, and the final performance score is averaged across all folds. This approach is consistent across all models and datasets, ensuring comparability of results.

A fixed random state was set for all cross-validation splits to ensure reproducibility. This means that any re-run of the experiments with the same configuration will produce identical fold assignments, eliminating a potential source of variance in the reported results.

4.6 Hyperparameter Tuning

To ensure a fair comparison across all models, hyperparameter tuning was applied to each model prior to evaluation. Without tuning, default parameters tend to favor certain

model families over others, potentially skewing the comparison in terms of both predictive performance and resource consumption [14].

Hyperparameter optimization was performed using Optuna [15], an open-source framework that employs Bayesian optimization to efficiently search the parameter space. Grid search was considered but deemed infeasible given the scale of experiments: assuming five hyperparameters per model with ten candidate values each, evaluated across three datasets and four models, a full grid search would require approximately 150,000 evaluations. Optuna was therefore configured to run 50 trials per model per dataset, striking a balance between tuning thoroughness and resource consumption, the latter being a relevant factor given that tuning runs contribute to the overall energy budget of this study, as described in Section 4.4.

Weighted F1-score was used as the optimization objective, as it accounts for class imbalance more robustly than accuracy, providing a more reliable estimate of model quality across datasets with varying class distributions.

Tuning was performed separately for each dataset, as the datasets differ substantially in size, feature space, and class distribution. A globally tuned model would not represent optimal performance on any individual dataset and would therefore introduce bias into the comparison.

Chapter 5

Implementation and Measurement

5.1 Metrics

idee nachdem ich schwartz et al gelesen habe seite: 60 FPO
While hardware-agnostic metrics such as Floating Point Operations (FPO) [1] would facilitate comparisons across different hardware configurations, direct energy measurement via CodeCarbon was preferred in this study as it captures actual resource consumption rather than a theoretical approximation. A limitation of this approach is that results are tied to the specific hardware used and may not generalize to other systems.

5.2 Hardware Setup

A critical decision in the experimental design was the selection of the computing platform on which all experiments would be conducted. Two options were considered: the university's shared computing infrastructure or a local machine. The latter was ultimately chosen, as it provides several key advantages for this study. First, it ensures complete reproducibility, since all experiments are executed on identical hardware configuration, eliminating interfering variables that could arise from shared resource conflicts or system variability. Second, it provides consistent availability and direct control over system configuration, which is particularly important given the extended runtime of the large-scale experiments (particularly on the 11M-instance HIGGS dataset). Third, local execution facilitates fine-grained performance monitoring and measurement of energy consumption, a primary objective of this study.

The hardware platform selected for all experiments is a consumer-grade desktop system with the following specifications: an NVIDIA RTX 4070 graphics processing unit (12 GB GDDR6 memory, 5,888 CUDA cores), an AMD Ryzen 7 5700X processor (8 cores / 16 threads, base clock 3.6 GHz), and 32 GB of DDR4 RAM. The system runs Windows 11 with all code implemented in Python 3.12.0.

The choice of hardware reflects a deliberate trade-off between experimental scope and practical feasibility. The RTX 4070 represents a mid-to-high-end consumer GPU with sufficient compute capability to execute GPU-accelerated training for XGBoost and MLP models in reasonable time, while remaining accessible to many practitioners. The Ryzen 7 5700X provides adequate CPU performance for baseline models (Logistic Regression, Random Forest) and CPU-based XGBoost training. Together, this configuration enables direct comparison of CPU- and GPU-accelerated training on the same platform, which is central to answering one of the research questions (*How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*).

It is important to note that energy consumption and CO2 emissions measured in this study are specific to this hardware configuration and the local electricity grid, and may not directly generalize to other systems or geographic regions. However, the relative performance comparisons (e.g., the energy efficiency ranking of different algorithms) are more likely to generalize, as they reflect algorithmic properties rather than hardware specifics alone.

5.3 Software Environment

All experiments were implemented in Python 3.12.0. The main libraries used are described below.

Scikit-learn [16] served as the foundation of the evaluation pipeline. It provided the `LogisticRegression` and `RandomForestClassifier` implementations, as well as `KFold` and `cross_validate` for cross-validation, `StandardScaler` for feature normalization, `make_pipeline` for chaining preprocessing and model steps, and `f1_score` and `make_scorer` for evaluation metrics.

The XGBoost library [10] was used for the gradient boosting model, as it provides a highly optimized implementation that supports both CPU and GPU training.

For the MLP, PyTorch [17] was used as the deep learning backend. To integrate the PyTorch model into the scikit-learn evaluation pipeline, the Skorch library [18] was used, which wraps PyTorch models in a scikit-learn-compatible interface.

Hyperparameter tuning for all models except Logistic Regression was performed using Optuna [15], which uses a tree-structured Parzen estimator to efficiently search the hyperparameter space.

Carbon emissions and energy consumption were tracked using CodeCarbon [19]. One known limitation is that CodeCarbon cannot directly read energy consumption from AMD processors and instead estimates it by multiplying the current CPU utilization with the processor's maximum thermal design power (TDP). This introduces some inaccuracy in the reported CPU energy values.

To ensure reproducibility, a fixed random seed of 42 was used for all data splits, cross-validation folds, and model initialization. A `requirements.txt` file is provided so

that the exact library versions used in this study can be reproduced.

Bibliography

[1] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Commun. ACM*, vol. 63, no. 12, pp. 54–63, Nov. 2020, ISSN: 0001-0782. DOI: 10.1145/3381831 Accessed: Mar. 19, 2026.

[2] E. Strubell, A. Ganesh, and A. McCallum, *Energy and Policy Considerations for Deep Learning in NLP*, Jun. 2019. DOI: 10.48550/arXiv.1906.02243 arXiv: 1906.02243 [cs]. Accessed: Mar. 19, 2026.

[3] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, *Quantifying the Carbon Emissions of Machine Learning*, Nov. 2019. DOI: 10.48550/arXiv.1910.09700 arXiv: 1910.09700 [cs]. Accessed: Mar. 19, 2026.

[4] M. F. Stefan Aeberhard, *Wine*, 1992. DOI: 10.24432/C5PC7J Accessed: Mar. 19, 2026.

[5] I-Cheng Yeh, *Default of Credit Card Clients*, 2009. DOI: 10.24432/C55S3H Accessed: Mar. 19, 2026.

[6] P. Baldi, P. Sadowski, and D. Whiteson, "Searching for Exotic Particles in High-Energy Physics with Deep Learning," *Nature Communications*, vol. 5, no. 1, p. 4308, Jul. 2014, ISSN: 2041-1723. DOI: 10.1038/ncomms5308 arXiv: 1402.4735 [hep-ph]. Accessed: Mar. 19, 2026.

[7] D. Whiteson, *HIGGS*, 2014. DOI: 10.24432/C5V312 Accessed: Mar. 19, 2026.

[8] D. R. Cox, "The Regression Analysis of Binary Sequences," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958, ISSN: 0035-9246. JSTOR: 2983890. Accessed: Apr. 8, 2026.

[9] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, ISSN: 1573-0565. DOI: 10.1023/A:1010933404324 Accessed: Mar. 19, 2026.

[10] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16, New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794, ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785 Accessed: Mar. 19, 2026.

[11] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, "Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning," *Sustainable Computing: Informatics and Systems*, vol. 38, p. 100 857, Apr. 2023, ISSN: 22105379. DOI: 10.1016/j.suscom.2023.100857 Accessed: Apr. 10, 2026.

[12] T. Hastie, R. Tibshirani, and J. Friedman, "Model Assessment and Selection," in *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, T. Hastie, R. Tibshirani, and J. Friedman, Eds., New York, NY: Springer, 2009, pp. 219–259, ISBN: 978-0-387-84858-7. DOI: 10.1007/978-0-387-84858-7_7 Accessed: Apr. 10, 2026.

[13] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2*, ser. IJCAI'95, San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., Aug. 1995, pp. 1137–1143, ISBN: 978-1-55860-363-9. Accessed: Apr. 10, 2026.

[14] A. Bagnall and G. C. Cawley, *On the Use of Default Parameter Settings in the Empirical Evaluation of Classification Algorithms*, Mar. 2017. DOI: 10.48550/arXiv.1703.06777 arXiv: 1703.06777 [cs]. Accessed: Apr. 10, 2026.

[15] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19, New York, NY, USA: Association for Computing Machinery, Jul. 2019, pp. 2623–2631, ISBN: 978-1-4503-6201-6. DOI: 10.1145/3292500.3330701 Accessed: Apr. 1, 2026.

[16] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011, ISSN: 1533-7928. Accessed: Mar. 19, 2026.

[17] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Dec. 2019. DOI: 10.48550/arXiv.1912.01703 arXiv: 1912.01703 [cs]. Accessed: Mar. 19, 2026.

[18] B. Ghosh, *Skorch: The Bridge Between PyTorch and Scikit-Learn*, Sep. 2024. Accessed: Mar. 26, 2026.

[19] *Mlco2/codecarbon*, CodeCarbon, Mar. 2026. Accessed: Mar. 19, 2026.

List of Figures

List of Tables